

Condicionales y bucles en Python

~~Sesión 2~~ - Introducción a la programación

Docente: José Zevallos



Objetivos de aprendizaje

Al finalizar la sesión, el estudiante:

- comprende cómo tomar decisiones con `if`, `elif` y `else`;
- reconoce operadores de comparación, lógicos y de pertenencia;
- utiliza bucles `for` y `while` para automatizar tareas repetitivas;
- resuelve ejercicios básicos de clasificación, sumatoria y construcción de listas.



Ruta de la sesión

01.

Condicionales

02.

Operadores
lógicos

03.

Bucles for
y while

04.

Práctica
aplicada



01.

 **Condicionales**



¿Para qué sirve un condicional?

Un condicional permite ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.

Estructura general:

- ▶ `if`
- ▶ `elif`
- ▶ `else`

Ejemplo conceptual:

- ▶ si la nota es mayor o igual a 11, el estudiante aprueba;
- ▶ en caso contrario, no aprueba.

Esto convierte reglas lógicas en instrucciones programables.

Operadores comunes

Para construir condiciones se utilizan operadores como:

Comparación

- ▶ ==, !=, <, <=, >, >=

Lógicos

- ▶ and, or, not

Pertenencia

- ▶ in

Estos operadores permiten formular reglas más complejas y precisas.

Ejemplo de clasificación

Un caso frecuente es clasificar un valor.

- ▶ si $x < 0$, entonces el número es negativo;
- ▶ si $x == 0$, entonces es cero;
- ▶ si $x > 0$, entonces es positivo.

Este patrón aparece en muchos problemas reales:

- ▶ validar datos;
 - ▶ asignar etiquetas;
 - ▶ controlar decisiones dentro de un programa.
-

02.

 **Bucles**



Bucle for

El bucle `for` recorre una secuencia o colección.

Ejemplos comunes:

- ▶ recorrer una lista;
- ▶ repetir una acción un número fijo de veces;
- ▶ usar `range(n)` para generar una secuencia.

Ejemplo:

- ▶ sumar los números de 0 a 4.

El `for` es útil cuando se conoce la colección o la cantidad aproximada de repeticiones.

Bucle while

El bucle `while` repite instrucciones mientras una condición siga siendo verdadera.

Se usa cuando la repetición depende de una condición y no necesariamente de una cantidad fija de pasos.

Ejemplo clásico:

- ▶ calcular un factorial;
- ▶ continuar mientras $n > 1$.

Precaución importante:

- ▶ siempre debe existir un cambio en la condición para evitar bucles infinitos.
-

¿Cuándo usar for y cuándo usar while?

Usar for cuando:

- ▶ se recorre una lista;
- ▶ se itera sobre `range()`;
- ▶ se conoce la estructura a recorrer.

Usar while cuando:

- ▶ la repetición depende de una condición;
 - ▶ el proceso debe continuar hasta que ocurra un cambio;
 - ▶ el número de iteraciones no está fijado desde el inicio.
-

03.

 **Práctica guiada en notebook**



Ejercicios propuestos en clase

Ejercicio 1:

- ▶ clasificar un número como negativo, cero o positivo.

Ejercicio 2:

- ▶ usar un `for` para sumar solo los números pares entre 1 y 20.

Ejercicio 3:

- ▶ calcular $6!$ usando `while`.

Ejercicio 4:

- ▶ a partir de una lista dada, construir otra con los cuadrados de sus elementos.
-

Errores frecuentes que debemos evitar

Al iniciar en programación suelen aparecer estos errores:

- ▶ olvidar los dos puntos : al final de `if`, `for` o `while`;
- ▶ problemas de indentación;
- ▶ usar `=` en lugar de `==` para comparar;
- ▶ crear un `while` sin actualizar la variable de control;
- ▶ confundir recorrer una lista con recorrer posiciones numéricas.

Detectar estos errores temprano es parte del aprendizaje.

04.

✓ **Cierre de la sesión**



Ideas de cierre

En esta sesión trabajamos dos ideas centrales de la programación:

- ▶ decidir qué hacer según una condición;
- ▶ repetir instrucciones para automatizar tareas.

Puntos clave:

- ▶ `if` permite ramificar la lógica;
- ▶ `for` permite recorrer secuencias;
- ▶ `while` permite repetir mientras una condición sea verdadera.

Con variables, colecciones, condicionales y bucles, ya se tiene una base suficiente para empezar a resolver problemas sencillos con Python.
